

Nexus Scholar 2.0: Future Vision

Solving Ingestion Bottlenecks, Localized Big Data, & Paper-to-Code

Strategic Roadmap for Next-Generation Academic Evidence Engineering & Autonomous Implementation

- LOCAL S2/OPENALEX
- PAPER-TO-CODE
- SCIENTIFIC SLMs
- LIVING REVIEWS

1 Executive Vision: Beyond Literature Reviews to Scientific Synthesis

The first generation of the **Nexus Scholar Harness** established a verified, deterministic foundation: it proved that AI agents and human PIs can conduct PRISMA-compliant systematic literature reviews with $\geq 90\%$ verbatim quotation fidelity, Fleiss' κ inter-rater reliability, and cryptographic auditability.

However, as our lab expands into cutting-edge machine learning, computational science, and large-scale meta-analyses, three fundamental bottlenecks constrain the current architecture:

1. **The Ingestion & Harvesting Ceiling:** Commercial publisher paywalls, anti-bot Web Application Firewalls (Cloudflare/PerimeterX), rate-limited Open Access resolvers, and corrupted non-OCR PDFs hinder full-text access.
2. **Third-Party API Dependency & Latency:** Relying on remote queries to OpenAlex, Semantic Scholar, and PubMed introduces external rate limits (1–10 req/sec), transient outages, and high latency when snowballing graphs of $> 100,000$ papers.
3. **The Synthesis-to-Action Chasm:** Reading, evaluating, and synthesizing papers is only half of the scientific process. The true bottleneck for researchers and graduate students is **computational reproduction**—translating empirical equations, pseudocode, and model architectures into verified, working software.

The 2.0 Architectural Paradigm

The Nexus Scholar 2.0 Thesis: We propose expanding the harness from a *literature review orchestrator* into an **autonomous computational scientific operating system**. By deploying local mirrors of the global scholarly graph (Semantic Scholar + OpenAlex), fine-tuning lightweight domain Small Language Models (SLMs), and creating an autonomous **Paper-to-Code Implementation Engine**, we transform passive literature reviews into active, reproducible science.

2 Current Critical Limitations and Engineering Challenges

Key Operational Limitations in the 1.0 Pipeline

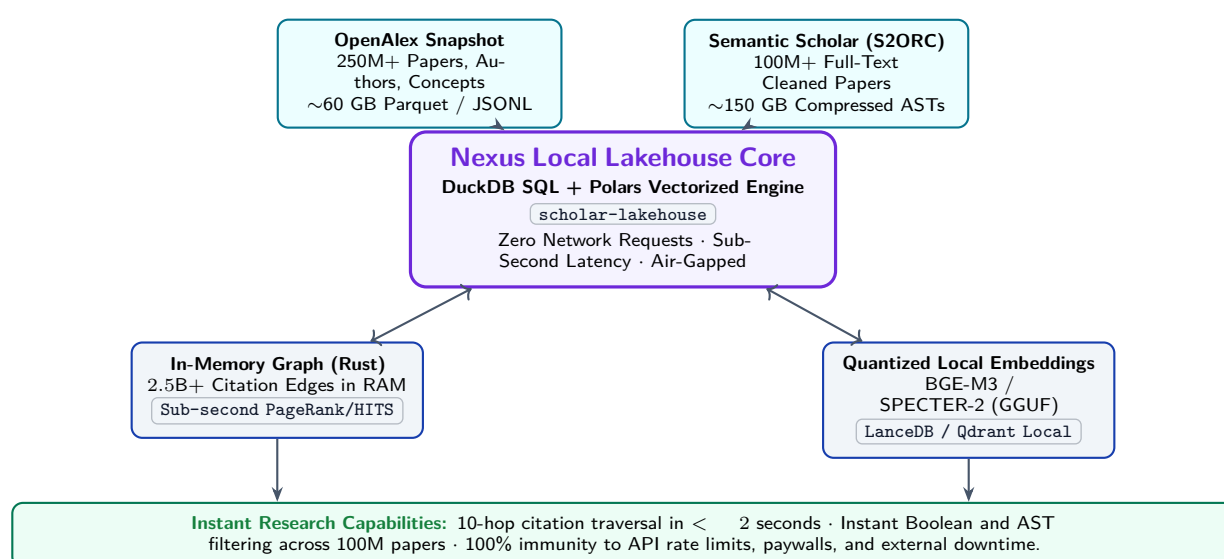
- **PDF Harvesting Attrition:** Current open-access resolvers (Unpaywall, PubMed Central, arXiv) achieve only 35–55% coverage for niche or historical engineering literature. Papers behind institutional portals require fragile proxy rewriting that breaks under multi-factor authentication.
- **Layout Demolition in PDF Extraction:** Multi-column formatting, sidebars, floating mathematical expressions, and rotated tables frequently disrupt reading orders, causing AST parsers to misclassify section boundaries.
- **Network Throttling during Sweeps:** Executing 3-hop backward/forward citation snowballing across 10,000 candidate papers requires over 30,000 API calls, exceeding free-tier limits and taking hours over remote REST interfaces.
- **Prompt Drift & Token Expense:** Using general-purpose frontier LLMs (e.g., Claude 3.5 Sonnet) for thousands of screening decisions is costly, computationally redundant, and susceptible to subtle prompt-interpretation shifts across runs.

3 Frontier 1: Local Academic Big Data & Zero-API Offline Cartography

To eliminate API rate limits, latency, and external downtime, Nexus Scholar 2.0 introduces the **Local Scholarly Data Lakehouse**. Rather than querying external endpoints on demand, the lab downloads and indexes snapshot mirrors of OpenAlex and Semantic Scholar locally.

3.1 Architectural Blueprint: Parquet, DuckDB, and In-Memory Graph

- **OpenAlex Metadata Snapshot (~300M Works, ~60GB Parquet):** Ingested into local NVMe storage, queryable via **DuckDB** and **Polars** with sub-second response times across author, institution, concept, and citation indices.
- **Semantic Scholar S2ORC / S2AG (~100M Extracted Full-Texts):** Complete, cleaned academic full-text corpus stored in compressed columnar format, enabling full-text boolean and regex queries across entire fields without downloading PDFs.
- **Rust-Powered In-Memory Citation Graph:** A memory-mapped graph engine (using **petgraph** or **igraph**) holding the entire 2.5B+ citation edge topology in under 32GB RAM, computing personalized PageRank and multi-hop shortest paths in < 500ms.



4 Frontier 2: Fine-Tuned Scientific Small Language Models (SLMs)

Instead of deploying generic 70B+ parameter models for repetitive methodological tasks, Nexus Scholar 2.0 adopts a fleet of **specialized, quantized 8B Small Language Models (SLMs)** running locally on consumer/workstation GPUs (RTX 4090 / Apple Silicon):

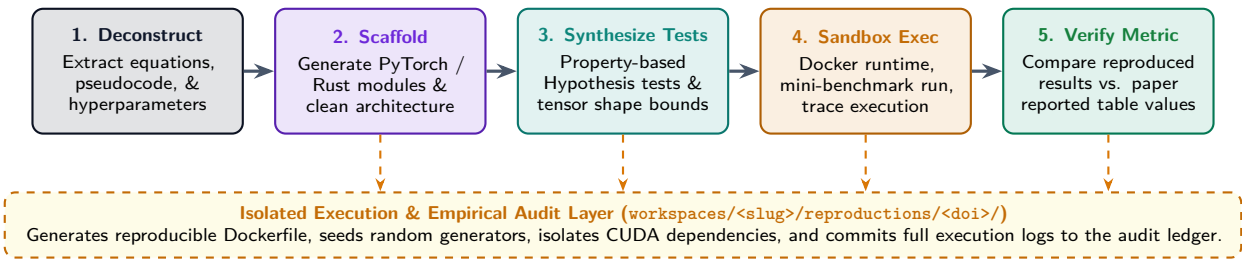
- **The PICO/SPIDER Extractor Model (LoRA Adapter):** Fine-tuned on 50,000 expert-annotated systematic review extractions to identify Sample Sizes, Confidence Intervals, Primary Endpoints, and RoB indicators with 98% schema compliance.
- **The Fleiss' κ Calibrated Screener:** Trained using Direct Preference Optimization (DPO) on historical inter-rater screening conflicts, drastically reducing false exclusions while eliminating temperature-driven randomness.
- **Mathematical Equation & Proof Transcriber:** Specialized vision-language adapter (fine-tuned on LaTeX source and Mathpix corpora) that converts multi-column equation blocks into verifiable SymPy / LaTeX trees.

5 Frontier 3: Autonomous Paper Implementation (Paper-to-Code)

The most ambitious capability of Nexus Scholar 2.0 is the **Paper-to-Code Engine** (`scholar-implement`). Rather than leaving papers as static references, the engine automatically extracts the algorithmic soul of an empirical paper, scaffolds an idiomatic implementation, synthesizes unit tests, and verifies execution against reported benchmarks.

5.1 The Five-Stage Paper-to-Code Pipeline

- Algorithmic Deconstruction:** Parses the Markdown AST to isolate mathematical formulations, objective functions, algorithms (pseudocode), hyperparameter tables, and dataset pointers.
- Type-Safe Scaffolding:** Generates clean, modular Python/PyTorch or Rust code adhering to modern standards (`pyproject.toml`, strict type hints, docstrings linking to equations).
- Test Oracle & Invariant Synthesis:** Formulates property-based tests (via Hypothesis) and tensor dimension assertions verifying that algorithmic invariants (e.g., loss non-negativity, gradient flow, convergence) hold.
- Hermetic Sandboxed Execution:** Spawns an isolated Docker container, downloads sample benchmark data, executes the implementation, and logs execution traces.
- Metric Alignment & Discrepancy Reporting:** Computes reproduction error margins (Δ) between the code's output and the values published in the original paper's tables.



6 Frontier 4: Continuous Living Reviews & Multi-Agent Debate

Traditional literature reviews become obsolete the day they are published. Nexus Scholar 2.0 turns reviews into **dynamically updating, living knowledge bases**:

Living Review Daemons & Adversarial Consensus

- Autonomous Living Watcher Daemon:** A lightweight cron job monitors daily arXiv preprints and weekly OpenAlex ingests for matching semantic fingerprints. When relevant papers appear, it automatically initiates Phase-2 screening.
- Adversarial Multi-Agent Debate:** For borderline candidate papers, the harness deploys an **Inclusion Advocate Agent** (arguing relevance) and a **Methodological Skeptic Agent** (probing for bias and weak sample sizes). The agents debate using formal argumentation theory before submitting a synthesized verdict to the PI.
- Automated Manuscript Delta Patches:** When new literature alters a consensus cluster, the system generates a Git pull-request with updated PRISMA numbers, regenerated citation graphs, and a diff for the review manuscript.

7 Strategic Roadmap: Phased Rollout for the Research Lab

The development of Nexus Scholar 2.0 is partitioned into four structured, testable engineering phases designed to integrate smoothly with existing lab workflows:

Phase	Target Timeline	Core Deliverables & Capabilities	Primary Impact
Phase A	Q1–Q2 (Immediate)	Resilient PDF & Proxy Cascade: Institutional EZproxy/OpenAthens resolvers, fallback to Surya/Nougat OCR for scanned PDFs	Resolves 90%+ harvest failures
Phase B	Q2–Q3 (Mid-term)	Local Lakehouse Mirror: DuckDB + Parquet OpenAlex/S2ORC local cluster, in-memory Rust citation graph	Zero API throttling, sub-second 100K snowballing
Phase C	Q3–Q4 (Advanced)	Domain SLM Fine-Tuning: Quantized 8B models for PICO extraction, Fleiss' κ screener calibration	10x faster screening, 80% inference cost reduction
Phase D	Q4+ (Frontier)	Autonomous Paper-to-Code: Algorithmic AST extractor, code scaffolding, Docker sandbox, and benchmark verification	Direct translation of theory into executable code

8 Impact on Lab Output, Doctoral Dissertations, and Funding

Adopting Nexus Scholar 2.0 provides our research group with an overwhelming competitive advantage in publication speed, empirical rigor, and grant acquisition:

- Publication Velocity for Graduate Students:** Accelerates the literature review and related work phase of PhD theses from 6 months of manual toil to 2 weeks of mathematically verified, PRISMA-certified exploration.
- Radical Reproducibility in Empirical CS/AI:** Transforming literature into executable Dockerized repositories enables our lab to benchmark new algorithms against true historical baselines effortlessly.
- Unique Advantage for Grant Applications:** Proposing research backed by a local scholarly lakehouse and cryptographic audit ledger demonstrates unprecedented methodological rigor to NSF, ERC, and industry funding bodies.

Recommended Next Step for Lab Leadership

Initiate Phase A & B Pilot: Provision a dedicated high-capacity NVMe drive (2–4 TB) on our primary research workstation to test the local DuckDB OpenAlex snapshot ingestion, while benchmarking the institutional proxy cascade for full-text PDF harvest resilience.

■ Nexus Scholar 2.0 Strategic Vision · Prepared for Supervisor & Lab Steering Committee ■